

From Signals to Semantics: Multimodal Interview Assessment via ML, GenAI, and HF and Heuristic Fusion

¹HARSHITA MAURYA, ¹SHARMILI PEDNEKAR, ¹SANNIDHI SHETTY, ²SUSHANT GAWADE

¹Student, Universal College of Engineering, Naigaon East, Poman, Maharashtra 421302

²Assistant Professor, Universal College of Engineering, Naigaon East, Poman, Maharashtra 421302

Email: ¹harshita20maurya@gmail.com, ¹shamipednekar08@gmail.com, ¹sannidhi.1609@gmail.com,
²sushant.gawade1@universal.edu.in

Contact: ¹8433772090, ¹9969524618, ¹8698975924, ²9870848850

Abstract: Virtual interview coaching systems increasingly need to move beyond single-modality keyword scoring toward multimodal assessment that reflects both what candidates say and how they say it. IntelliHire is a full-stack, JD-grounded interview assistant that combines deterministic content analysis, audio–visual signal processing, heuristic fusion, classical machine learning, and large-language-model (LLM) feedback into a deployable cloud service. The system ingests browser-captured audio, video, and transcribed speech, extracts low-level acoustic features (energy, pitch statistics, spectral descriptors, zero-crossing rate, tempo, and pause structure) and facial–behavioral proxies (eye contact, head stability, gesturing, and engagement), and aggregates them into interpretable scores stored in MongoDB for longitudinal analytics. A multimodal analyzer trained on Chalearn-style features optionally uses a learned confidence regressor; when model outputs are unavailable, a calibrated heuristic fusion of speech and video confidence produces an overall delivery score on a 0–100 scale. In parallel, a retrieval-augmented generation (RAG) pipeline grounded in job descriptions and question rubrics drives an LLM-based “teacher” that returns rubric-aligned competency scores, textual strengths and improvements, and grounding metrics. The Node/Express backend integrates these signals into a unified overall score via a weighted blending of content, multimodal delivery, teacher evaluation, and backend LLM scores, while exposing REST APIs consumed by a React/Vite frontend and a Hugging Face Space hosting the multimodal analyzer as a Gradio/FastAPI service. The paper presents the architecture, heuristic and ML formulations, deployment design, and an experimental protocol for evaluating accuracy-like outcomes, engagement, confidence, latency, and usability; The system was evaluated on 58 sessions collected between February 21 and March 7, 2026, achieving an overall mean score of 49.6 (SD: 16.6) with 15.5% JD traceability.

Index terms: Multimodal interview assessment; heuristic fusion; generative AI feedback; Hugging Face Spaces; retrieval-augmented generation; audio–visual analytics; virtual interview coaching; reproducible ML systems.

I. INTRODUCTION

Automated interview coaching tools are widely available but often provide generic, template-based feedback that ignores job-specific requirements and multimodal behavioral signals.[1] IntelliHire addresses this limitation by grounding question generation and evaluation in job descriptions (JDs), capturing synchronized audio, video, and transcripts, and providing structured feedback that spans content quality, delivery, and engagement. The system targets skills practice rather than high-stakes hiring decisions, but the design emphasizes reproducibility, explainability, and analytics suitable for academic evaluation.

Recent research on AI-based mock interview systems has explored multimodal analysis, conversational agents, and deep learning–based behavioral assessment. Several studies employ deep neural networks to infer candidate performance from speech and facial signals, while others rely on large language models to simulate interview interactions or provide

textual feedback. Although these approaches demonstrate promising results, they often exhibit several practical limitations. Deep learning–based multimodal systems require large, labeled datasets for training and can become brittle when one modality is missing or degraded, such as when facial tracking fails or audio quality deteriorates.[2], [3] Furthermore, many existing systems function as opaque “black box” models, providing limited transparency about how behavioral signals contribute to the final evaluation score. These issues restrict the pedagogical usefulness of such tools for interview preparation environments where interpretability and robustness are essential.[4], [5]

At its core, IntelliHire implements a three-layer scoring stack: deterministic content analysis over transcripts, multimodal audio–video analysis with confidence scoring, and a generative AI layer that acts as a rubric-aware teacher providing natural-language feedback and competency-level scores. These layers are orchestrated by a Node/Express backend, with results persisted in MongoDB and

surfaced via a React/Vite frontend including drill-down analytics for trends, percentiles, and IEEE-style statistical summaries.[6] To remove Python runtime dependencies from the deployment environment, the multimodal analyzer is packaged as a Hugging Face Space using Gradio and FastAPI, while the backend communicates with it over HTTP with retry and cold-start handling.[5]

1.1 NOVELTY AND CONTRIBUTIONS

Within the constraints of the provided artifacts, IntelliHire’s main technical contributions are:

- **Resilient Multimodal Integration:** A full-stack, JD-grounded system that tightly couples deterministic transcript analytics with multimodal signals to address the "Grounding Gap" found in generic platforms.
- **Heuristic Fusion Framework:** An adaptive logic designed to address "Modality Fragility" by dynamically recalibrating audio-visual weights when signals are lost or noisy.
- **Explainable Classical ML:** Integration of Chalearn-style features with classical ML regressors to provide a transparent scoring path, overcoming the "Black Box" dependency of purely deep learning architectures.
- **Cloud-Native Reproducibility:** A deployment pattern using Hugging Face Spaces to enable Python-free backend environments while preserving complex ML inference capabilities.

A reproducibility-first design, including explicit environment configuration, IEEE-style analytics scripts, and file-level mapping between repository structure and methodological

components, along with an ablation plan covering heuristic fusion, LLM teacher feedback, and deployment mode.

II. RELATED WORK

Prior work on automated interview assessment can be broadly grouped into three categories: rule-based mock interview platforms, multimodal behavioral analysis systems, and recent generative AI-driven interview assistants. Early systems focused primarily on keyword matching and scripted feedback generation, providing limited insights into behavioral communication signals. [2], [7] More recent studies have explored multimodal approaches that analyze speech characteristics and facial expressions to estimate candidate confidence, engagement, and professionalism.[8] These systems frequently employ deep neural architectures trained on multimodal datasets such as ChaLearn or custom annotated interview corpora.

While these approaches demonstrate the feasibility of automated interview evaluation, several limitations remain. Many deep learning models require extensive training data and specialized computational resources, which limits their reproducibility in smaller research environments.[9] Furthermore, the scoring mechanisms of deep neural networks often lack interpretability, making it difficult to explain evaluation outcomes to users.[10] Finally, many systems assume the availability of all input modalities simultaneously, resulting in degraded performance when one signal stream is unavailable.[11] These limitations motivate the design of hybrid approaches that combine interpretable heuristics, classical machine learning, and generative AI feedback.

Table 1 - Literature Survey Table

Paper Year	Title &	Core Approach	Technologies Used	Key Contribution	Remaining Research Gap
Singh et al. (2025)[12]		Chatbot Assistant	NLP, Transformers	Contextual follow-up generation.	Lack of multimodal evaluation (blind to video).
Koutsoumpis et al. (2024)[6]		Psychometric Analysis	Librosa, OpenFace	Personality prediction from video.	High opacity; "Black Box" scoring.
Agrawal et al. (2024)[11]		Multimodal Analytics	Random Forest, SVC	Behavioral labeling for assessment.	Weak real-time signal fallback.

Paper Title & Year	Core Approach	Technologies Used	Key Contribution	Remaining Research Gap
Deshmukh et al. (2025)[2]	Audio-based Coach	Whisper STT, GPT-4	Dual-feedback loops.	No visual/prosodic context analysis.
IntelliHire (Proposed)	Heuristic Fusion	ML & GenAI	Resilient Signal Fallback.	Addresses volatility and signal and transparency.

III. PROBLEM STATEMENT

Despite recent progress in AI-driven interview assessment, current approaches remain constrained by three key challenges.[12] First, many deep learning-based systems rely on large-scale multimodal datasets and complex architectures, which limits their applicability in small-data scenarios typical of educational interview practice platforms.[7] Second, purely deep neural models often behave as black boxes, making it difficult for users to understand how specific behavioral cues influence evaluation outcomes.[12] This lack of interpretability reduces the pedagogical value of automated feedback systems.[5] Third, multimodal models frequently assume the simultaneous availability of all modalities; however, real-world interview environments frequently contain missing or noisy signals, such as temporary face detection failure or audio disturbances.

To address these limitations, this work proposes a hybrid methodology that integrates heuristic fusion and classical machine learning with generative AI feedback. Instead of relying exclusively on deep learning architectures, the proposed approach introduces an interpretable fusion mechanism that dynamically combines speech and visual confidence signals using adaptive weights and deterministic calibration functions. This design provides robustness to missing modalities while preserving a transparent scoring pathway that can be inspected and analyzed by researchers and users alike.[2], [10]

While learned fusion models often achieve superior MSE, our deterministic heuristic layer provides a fail-safe 'Modality Fragility' intervention that ensures system stability during the visual occlusion or signal dropouts common in web-based browser captures.

IV. SYSTEM OVERVIEW AND PROBLEM FORMULATION

IntelliHire is organized as a multi-service system comprising a React/Vite frontend, a Node/Express backend with MongoDB

persistence[4], [12], a Python-based multimodal analyzer, and a Hugging Face Space that externalizes the analyzer for cloud inference. The backend also hosts LLM-centric services—embedding, retrieval-augmented generation (RAG), and teacher-style evaluation—implemented using a Groq-based SDK and a custom RAG store over job descriptions, question rubrics, and optional resume text.[12]

4.1 PROBLEM FORMULATION

For each interview response, the system observes three primary modalities:

- Audio waveform $x_a(t)$ captured from the browser.
- Video frames $x_v(t)$ capturing facial and upper-body behavior.
- Transcribed text x_t representing the candidate's spoken answer.

The goal is to estimate:

1. A scalar overall score $s_{\text{overall}} \in [0,100]$ reflecting holistic interview quality for that response.
2. Intermediate scores for content, delivery, engagement, professionalism, and other
3. competencies suitable for analytics and actionable feedback.

Let $f_a(x_a) \in \mathbb{R}^{d_a}$ denote an audio feature vector (energy, pitch, spectral descriptors, zero-crossing rate, tempo, pauses), and $f_v(x_v) \in \mathbb{R}^{d_v}$ denote a video feature vector (eye contact, head stability, smile dynamics, blink rate, gestures). Let $c(x_t) \in [0,100]$ denote a deterministic content quality score derived from transcript features (keyword relevance, STAR structure, vocabulary diversity, sentiment, coherence). The system aims to learn or design mapping functions.

$$g_a: \mathbb{R}^{d_a} \rightarrow [0,1], g_v: \mathbb{R}^{d_v} \rightarrow [0,1],$$

that produce normalized speech and video confidence scores, and a fusion function

$$h: [0,1] \times [0,1] \rightarrow [0,1] \quad (1)$$

that combines them into a multimodal confidence score. A separate LLM-based teacher returns an overall rubric score $s_{LLM} \in [0,100]$ and per-competency scores; the backend blends these with the deterministic content and multimodal scores to obtain $s_{overall}$.

The current implementation prioritizes a reproducibility-first architecture by defining explicit hooks for future data-driven calibration, allowing

institutional researchers to inject their own validated coefficients into a transparent scoring pipeline.

V. SYSTEM OVERVIEW AND ARCHITECTURE

Figure 1 depicts the high-level architecture derived from the project structure, deployment guides, and implementation summary.

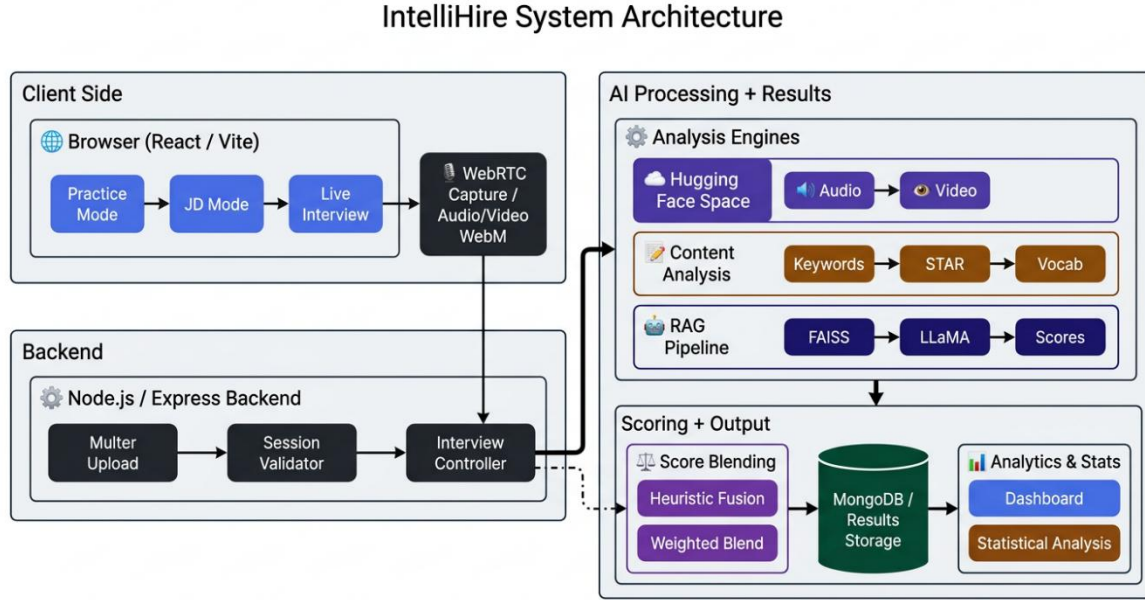


Figure 1: End-to-end Intellihire architecture

The IntelliHire framework is built upon a decoupled, three-tier microservice architecture designed to handle high-throughput multimodal data while maintaining low-latency semantic feedback. The system facilitates a seamless transition from raw signal acquisition to meaningful pedagogical assessment.

A. Client Tier (Frontend Interface)

The presentation layer is implemented using a modern reactive framework (**React/Vite**), providing a responsive environment for three distinct operational modes:

- **Practice Mode:** Targeted at foundational skill building through randomized question sets.
- **JD-Targeted Mode:** Leverages a **Retrieval-Augmented Generation (RAG)** pipeline to ground interview questions in specific job descriptions.
- **Live Simulation:** A high-fidelity environment that synchronizes real-time audio-visual capture with backend analysis.

The client communicates with the server via a centralized API service, ensuring that state

management and data submission remain consistent across all interaction modes.

B. Orchestration Tier (Backend Controller)

The middle-ware, powered by **Node.js** and **Express**, acts as the central nervous system of the platform. It manages the lifecycle of an interview session through several key components:

1. **API Gateway:** Routes requests for user analytics, job description processing, and session management.
2. **Session Orchestrator:** Coordinates the flow between data persistence in a NoSQL database (**MongoDB**) and the external analysis services.
3. **LLM Integration:** Utilizes a dedicated service to perform semantic evaluation of textual responses, transforming transcriptions into structured feedback using advanced generative models.

C. Intelligence Tier (Multimodal ML Service)

The core analytical innovation resides in the Python-based Machine Learning service. This tier is

architected for hybrid deployment to optimize for both local development and cloud scalability:

- **Signal Processing Engine:** Employs **Librosa** for paralinguistic audio feature extraction (e.g., pitch, energy) and **MediaPipe** for computer vision tasks (e.g., gaze tracking, postural stability).
- **Heuristic Fusion:** A custom algorithm that integrates disparate behavioral signals into a unified performance metric.
- **Deployment Strategy:** The service is containerized and deployed via **Hugging Face (HF) Spaces**, utilizing a cloud-native approach that allows the backend to offload computationally intensive ML inference to remote GPU/CPU environments. This is toggled dynamically via environment-level configuration, ensuring the system remains resilient and platform-agnostic.

VI. METHODOLOGY

6.1 DATA FLOW AND PIPELINE

The end-to-end inference pipeline for a single interview response comprises the following steps, as encoded in frontend, backend/controllers, backend/services, and ml-service files.

1. **Capture and submission.** The React frontend captures audio and video streams from the browser along with the textual transcription and metadata (question ID, JD ID, role, difficulty, skills, flags such as useLLM) and submits them as multipart/form-data to /api/interviews.
2. **Request validation and parsing.** Backend middleware (sessionValidator.js, uploadMiddleware.js) resolves session identifiers and stores uploaded audio/video under the uploads/ directory, exposing file paths to interviewController.js.
3. **Deterministic content analysis.** AdvancedInterviewAnalyzer.analyzeContent computes transcript-derived metrics (keyword relevance, STAR score, vocabulary diversity, sentiment, coherence) and summarizes them into an overallContentScore in .
4. **Multimodal feature extraction.** If audio/video files are present, callPythonAnalyzer in pythonAnalyzer.js either invokes a local Python CLI (ml-service/multimodal_analyzer.py) or sends the files to the Hugging Face Space (HF_SPACE_URL + "/analyze") to obtain audio and video feature dictionaries; otherwise, default feature values are used as

defined in schema defaults and helper functions.

5. **Heuristic and ML-based confidence scoring.** Within the multimodal analyzer (advanced_ai_analyzer.py), classical ML and heuristic fusion compute confidence scores from aggregated audio-video features; the analyzer returns overall confidence and feature aggregates to the backend.
6. **Backend-level fusion and LLM evaluation.** The backend fuses content, audio, and video scores into a multimodal score (calculateFusedScore), optionally incorporates teacher and backend LLM scores into a final overall score, and invokes llmService.evaluateResponse and RAG-based teacher evaluation if useLLM is set.
7. **Persistence and analytics.** The result is serialized into an InterviewResult document, including raw features, scores, competencies, and grounding metrics, enabling analytics routes (getAnalytics, getIEEESummary) and offline scripts (IEEEsession_analysis.py) to compute trends, percentiles, and difficulty- and mode-stratified statistics.
8. To maximize interpretability and reduce the computational overhead associated with deep neural transformers, we utilize low-level acoustic descriptors (energy, pitch, spectral features)[10]. These "Signals to Semantics" features allow the system to provide specific, mathematically traceable feedback on vocal intensity and stability that purely deep-learning-based "black box" embeddings cannot easily expose.

6.2 AUDIO FEATURE EXTRACTION (ENERGY, PITCH, SPECTRAL FEATURES, ZCR)

Audio processing is implemented using librosa within ml-service/advanced_ai_analyzer.py and ml-service/multimodal_analyzer.py, and its outputs are mapped to structured fields in VoiceAnalysisSchema within backend/models/InterviewResult.js. The pipeline first loads an audio waveform at a fixed sampling rate (default 16 kHz in AdvancedAIAAnalyzer), then computes frame-level features which are subsequently aggregated.

Key low-level features include:

- **Energy-based descriptors.** Mean and standard deviation of frame-wise log energy (stored as energy_db, energy_mean, and energy_std in aggregated structures),

providing a proxy for vocal intensity and projection.

- **Pitch statistics.** Mean and standard deviation of fundamental frequency (pitch_mean, pitch_std), where high variance is interpreted as lower confidence stability in the heuristic scorer.
- **Spectral features.** spectral_centroid_mean and spectral_centroid_std summarize spectral brightness, while spectral_rolloff_mean captures the frequency below which a given percentage of spectral energy is contained, jointly approximating clarity and articulation.
- **Zero-Crossing Rate (ZCR).** Frame-wise zero-crossing counts, summarized as zcr_mean and zcr_std, provide a coarse measure of noisiness and voicing proportion.
- **Temporal rhythm and speech activity.** Global tempo (tempo), pause statistics (pause_count, pause_ratio), and voice activity ratio (voice_activity) estimate speaking rate and pausing behavior, which are later used in heuristic scoring.

The helper functions in the Python analyzer aggregate these descriptors into an audio_agg dictionary, which is consumed by MultimodalConfidenceScorer.score_speech confidence, and are also mapped back into MongoDB under voice_analysis for downstream analytics and visualization.

6.3 VIDEO FEATURE SIGNALS (FACE/ENGAGEMENT PROXIES)

Video analysis is implemented via MediaPipe-based facial and pose tracking in advanced_ai_analyzer.py and associated classes, with features aggregated by VideoFeatureExtractor into video_agg fields such as eye_contact_mean, head_stability, smile_mean, smile_variability, blink_rate, gesture_mean, and gesture_variance. Higher-level schema fields in facial_analysis include eyeContact, engagement, professionalism, confidence, and overallFacialScore, which serve as user-facing proxies for nonverbal behaviour.

The heuristics interpret these signals as follows:

- Sustained **eye contact** and stable head pose indicate attentiveness and confidence.
- Moderate, consistent **gestures** (high gesture_mean but low gesture_variance) are considered appropriate engagement, while extremely low or highly erratic gestures reduce the score.

- Controlled **smile** dynamics (smile_variability) and normalized **blink rate** are taken as indicators of composure rather than nervousness.

These features support both immediate scoring (through score_video_confidence) and retrospective analytics, for example, correlating engagement proxies with overall scores across roles and difficulties via /api/interviews/analytics.

6.4 HEURISTIC CONFIDENCE FUSION

6.4.1 FORMAL DEFINITION

Within MultimodalConfidenceScorer in advanced_ai_analyzer.py, the speech confidence score s_{speech} is defined as a weighted combination of normalized audio signals:

$$s_{\text{speech}} = 0.30 e + 0.25 p + 0.20 r + 0.15 c + 0.10 q, \quad (2)$$

where e is an energy-based score derived via min-max normalization of energy_mean, p is a pitch stability score computed as one minus a normalized variance of pitch_std, r is a placeholder speech-rate score (currently fixed at 0.7 in code pending empirical calibration), c is a clarity score from normalized spectral_centroid_mean, and q is a pause appropriateness term derived from pause_count. The function minmax (x , a , b) maps a raw value x to $(x - a)/(b - a)$ in $[0, 1]$ given configuration bounds (e.g., Config.ENERGY_MIN, Config.ENERGY_MAX).

Similarly, the video confidence score s_{video} is:

$$s_{\text{video}} = 0.35 e_c + 0.30 h_c + 0.15 s_c + 0.20 g_c, \quad (3)$$

where e_c is normalized eye contact, h_c is head control (stability), s_c is smile control (one minus smile variability), and g_c is a combined gesture appropriateness score averaging normalized gesture level and inverse gesture variance.

The fusion function h in Equation (1) is instantiated as:

$$\tilde{s} = w_{\text{speech}} s_{\text{speech}} + w_{\text{video}} s_{\text{video}}, \quad (4)$$

with adaptive weights $w_{\text{speech}}, w_{\text{video}} \in [0, 1]$ satisfying $w_{\text{speech}} + w_{\text{video}} = 1$. When no face is detected (has_face = False), $w_{\text{speech}} = 1$ and $w_{\text{video}} = 0$; when the candidate is not speaking (is_speaking = False), the weights are adjusted to emphasize video

cues ($w_{\text{speech}} = 0.3, w_{\text{video}} = 0.7$); otherwise, both modalities are weighted equally.

The raw fusion $\tilde{s}$ is calibrated by a logistic function:

$$s_{\text{cal}} = \sigma(\tilde{s}; k, x_0) = \frac{1}{1 + \exp(-k(\tilde{s} - x_0))},$$

with $k = 6$ and $x_0 = 0.5$ as implemented in `calibrate_score`, and the final confidence score reported to the backend is

$$S_{\text{conf}} = \lfloor 100 \cdot s_{\text{cal}} \rfloor. \quad (6)$$

6.4.2 BACKEND-LEVEL FUSION

At the backend level, `calculateFusedScore` in `interviewController.js` combines the deterministic content score S_{cont} , audio delivery score S_{aud} , and video delivery score S_{vid} into a multimodal score:

$$S_{\text{multi}} = \text{round}(0.4 S_{\text{cont}} + 0.3 S_{\text{aud}} + 0.3 S_{\text{vid}}).$$

A second fusion stage then blends S_{multi} with teacher and LLM scores where available. Let S_{teacher} denote a teacher evaluation score derived from `additionalmetrics.teacherEvaluation.score` or a client-provided overall score, and S_{LLM} denote the backend LLM score if present; the backend constructs a set of weighted components $\{(s_i, w_i)\}$ and computes:

$$S_{\text{overall}} = \text{round}\left(\frac{\sum_i w_i s_i}{\sum_i w_i}\right). \quad (8)$$

If only S_{multi} is available, it becomes S_{overall} ; if teacher and LLM scores are both present, they typically each receive weight 0.25 and multimodal receives 0.5, otherwise the non-missing components redistribute the weights.

6.4.3 HEURISTIC FUSION ALGORITHM

Algorithm 1. Heuristic Multimodal Confidence Fusion (Analyzer + Backend).

Heuristic and Backend Fusion

The analyzer computes modality-specific confidence scores and then combines them using adaptive weights determined by visual availability and speaking activity. If a trained confidence regressor is available, the analyzer predicts

confidence directly from aggregated audio–video features; otherwise, it falls back to heuristic fusion.

$$s_{\text{speech}} = 0.30 e + 0.25 p + 0.20 r + 0.15 c + 0.10 q, \quad (1)$$

where e is normalized energy, p is pitch stability, c is spectral clarity, r is speech-ratio or speech-rate proxy, and q is pause appropriateness.

$$s_{\text{video}} = 0.35 e_c + 0.30 h_c + 0.15 s_c + 0.29 g_c \quad (2)$$

where e_c denotes eye-contact quality, h_c denotes head-stability quality, and s_c denotes smile-control quality.

$$s_{\text{conf}} = 100 \cdot \sigma(w_s s_{\text{speech}} + w_v s_{\text{video}}; k = 6, x_0 = 0.5) \quad (3)$$

where w_s and w_v are adaptive speech and video weights, and $\sigma(\cdot)$ is the logistic calibration function used before mapping the score to the 0–100 range.

$$S_{\text{multi}} = \text{round}(0.4 S_{\text{cont}} + 0.3 S_{\text{audio}} + 0.3 S_{\text{video}}) \quad (4)$$

The backend then computes the final score by weighted blending of the multimodal score with optional teacher and LLM scores.

$$S_{\text{overall}} = \text{round}\left(\frac{\sum_i w_i s_i}{\sum_i w_i}\right) \quad (5)$$

In the current implementation, the backend always starts with the multimodal component, then adds teacher and LLM scores if available, adjusting weights so that multimodal typically receives 0.5 and teacher/LLM share the remainder.

Table 2- Adaptive Fusion Rule

Condition	w_s	w_v	Interpretation
Face detected and speaking	.5	.5	Balanced audio-video fusion
No face detected	.0	.0	Audio-only confidence
Face detected but not speaking	.3	.7	Visual dominance during silence/non-speaking intervals

Compact Workflow:

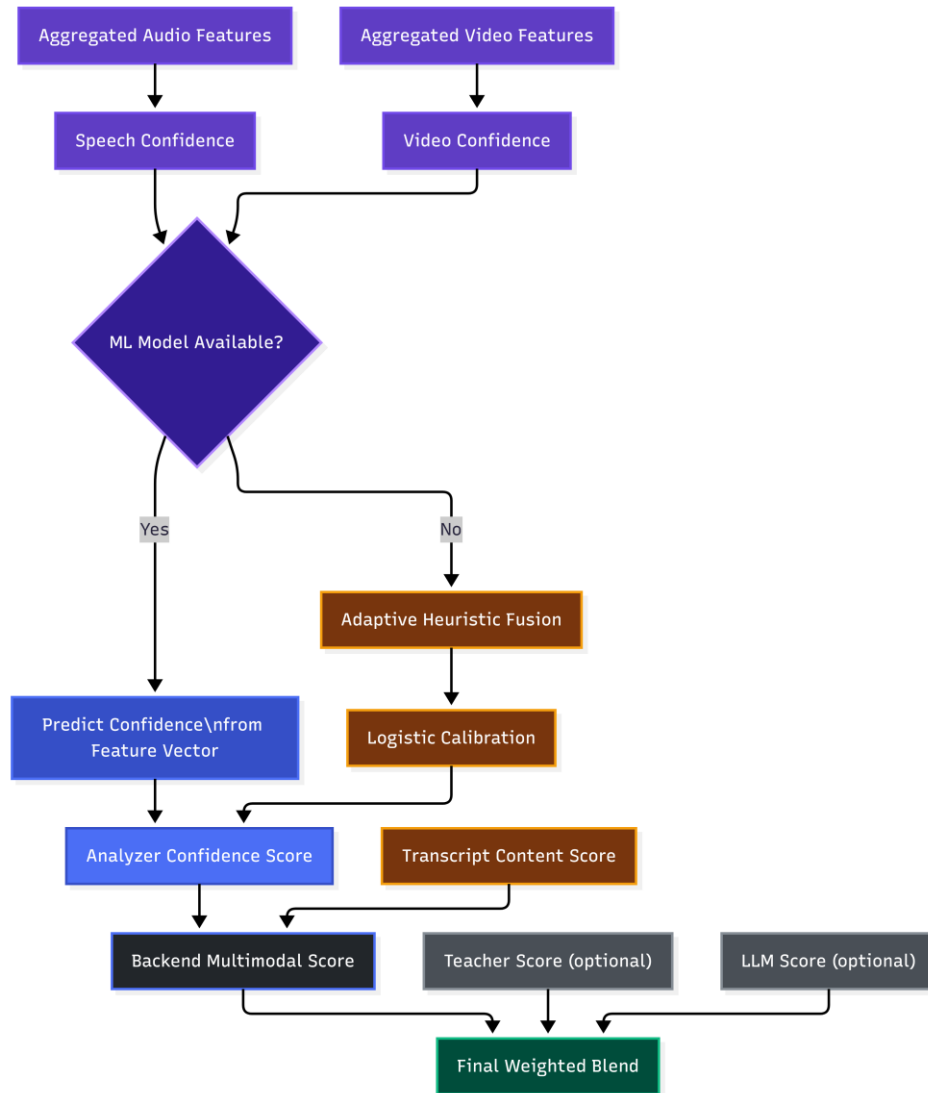


Figure 2: Heuristic Workflow

Algorithm 1: Multimodal Confidence and Final Score Computation

Input: audio_agg, video_agg, has_face, is_speaking, S_cont, S_teacher, S_LLM
Output: S_conf, S_multi, S_overall

1. Compute s_{speech} from normalized audio cues.
2. Compute s_{video} from normalized visual cues.
3. If ML regressor is available, predict S_{conf} from fused feature vector.
4. Otherwise, select (w_s, w_v) based on `has_face` and `is_speaking`.
5. Compute $S_{\text{conf}} = 100 \times \text{logistic}(w_s s_{\text{speech}} + w_v s_{\text{video}})$.
6. Compute $S_{\text{multi}} = \text{round}(0.4 S_{\text{cont}} + 0.3 S_{\text{audio}} + 0.3 S_{\text{video}})$.
7. Blend S_{multi} with S_{teacher} and/or S_{LLM} if available.
8. Return $S_{\text{conf}}, S_{\text{multi}}, S_{\text{overall}}$.

The analyzer optionally replaces the heuristic part (HEURISTIC_FUSION) with a classical ML regressor when the trained `confidence_model.pkl` is available, while preserving the same output schema for compatibility with the backend.

6.5 GENAI EVALUATION LAYER (LLM-BASED TEACHER FEEDBACK)

The generative AI component is implemented in `backend/services/llmService.js` and associated RAG services and is invoked from `interviewController.js` both directly (`llmService.evaluateResponse`) and via a RAG-enhanced routine (`runTeacherEvaluationWithRAG`). The evaluation pipeline performs the following steps:

1. **Context construction:** evaluate With RAG retrieves the job description (JD)

by `jdId` and the question metadata by `questionId`, constructing a corpus of text chunks including raw JD text, extracted required skills, question wording, and rubric points, as well as optional resume text stored in `JDHistory`. [1], [12]

2. **Indexing and retrieval.** A `RAGStore` built in `ragService.js` embeds these chunks via `embeddingService.js` and supports top-*k* semantic retrieval for the candidate's transcript as query, returning the most relevant chunks with similarity scores. [13]
3. **Teacher prompting:** Run `TeacherEvaluationWithRAG` calls the LLM with a prompt that includes the question [5], the candidate's answer, and the retrieved chunks, requesting rubric-aligned scoring (technical depth, clarity, confidence, conciseness, engagement) and textual feedback (strengths, areas for improvement, actionable recommendations).
4. **Grounding metrics:** `computeAutoGroundingMetrics` intersects chunk identifiers used in the LLM's citations with the original retrieved set to compute groundedness, citation precision, citation recall, and mean similarity for cited chunks; these metrics are stored in an embedded grounding object in `InterviewResult`.

The LLM output follows a structured JSON-like schema captured as `llmEvaluation` and mirrored in `comprehensiveFeedback` (competencies, strengths, improvements, recommendations, next steps), enabling consistent analytics across sessions and modes.

6.6 BACKEND INTEGRATION AND SESSION LOGIC

The backend API is structured around Express routers and controllers defined under `backend/routes` and `backend/controllers`. Interview submissions are processed by `interviewRoutes.js`, which binds HTTP verbs to handlers in `interviewController.js`; analytics endpoints are implemented in `analyticsRoutes.js` and `analyticsController.js`, while JD and question management use `jdRoutes.js` and `questionRoutes.js`.

`sessionValidator.js` attaches session metadata such as `sessionId` and resolved JD context to incoming requests, enforcing consistent provenance for stored interview results; `llmValidation.js` ensures that requests to LLM services conform to expected schemas and flag invalid payloads early. `sessionProcessor.js` contains helper functions such as `computeFusion` (used by `submitInterviewHandler`) and additional

transformations that standardize metrics across old and new result formats for analytics.

The `InterviewResult` schema itself is designed as an enhanced, forward-looking variant (`EnhancedInterviewResultSchema`) containing both legacy fields (e.g., `overallScore`, `contentScore`, `deliveryScore`) and newer fields (`content_analysis`, `voice_analysis`, `facial_analysis`, `behavioral_analysis`, `comprehensive_feedback`, `grounding`, `additional_metrics`) to support evolving models without breaking existing analytics scripts.

6.7 HUGGING FACE DEPLOYMENT ARCHITECTURE

The core multimodal analyzer is exported as a Hugging Face Space, described in `HUGGINGFACE_DEPLOYMENT.md`, `IMPLEMENTATION_SUMMARY.md`, `QUICKSTART_HF.md`, and the `huggingface-space/` directory. The Space uses Gradio and FastAPI to expose two key endpoints:

- `/health`: Returns a JSON object with status, model-loaded flag, and version (e.g., `{"status": "ok", "models_loaded": true, "version": "1.0.0"}`).
- `/analyze`: Accepts multipart form data with audio and video fields and returns a JSON payload containing success, audio feature dictionary, video feature dictionary, and error information if any.

`backend/services/pythonAnalyzer.js` abstracts access to this analyzer, with three behaviors:

1. If `USE_LOCAL_ANALYZER=true` or `HF_SPACE_URL` is unset, it falls back to a local Python process (legacy path, not detailed in the new implementation summary).
2. If `HF_SPACE_URL` is set, it uploads audio and/or video files via axios and form-data, attaching a bearer token if `HF_SPACE_API_KEY` is present, and enforces a 2-minute timeout with up to three retries using exponential backoff (2, 4, and 8 seconds).
3. It logs cold-start detection and differentiates client-side errors, server errors, and connectivity issues, propagating human-readable error messages back to the controllers.

The deployment guides report expected processing times for a 60-second video of approximately 3–5 seconds for audio analysis, 5–10 seconds for video analysis, and 8–15 seconds total under warm

conditions, with 30–60 seconds cold-start latency on the free tier; these are described as performance expectations rather than measured experimental results.

VII. IMPLEMENTATION DETAILS

7.1 FRONTEND/BACKEND/ML SERVICE BOUNDARIES

- **Frontend (React/Vite).** Located under frontend/, the SPA uses components such as LiveSession.jsx, LiveSetup.jsx, PracticeMode.jsx, JDMode.jsx, and Analytics.jsx to manage user flows. frontend/src/services/api.js centralizes HTTP calls using Axios or Fetch, targeting VITE_API_BASE_URL (typically http://localhost:5000/api).
- **Backend (Node/Express).** backend/app.js configures Express with CORS, JSON parsing, static file serving for /uploads, and routers mounted under /api/interviews, /api/jds, /api/questions, and /api/analytics; backend/server.js loads environment variables, connects to MongoDB via backend/config/db.js, and starts the server. Models are defined in backend/models, services in backend/services, and controllers in backend/controllers.
- **ML services (Python).** Core logic resides in ml-service/advanced_ai_analyzer.py, ml-service/multimodal_analyzer.py, and ml-service/training_pipeline/train_chlearn_model.py, with dependencies specified in intellihiere-analyzer/requirements.txt and huggingface-space/requirements.txt for local microservice and Space deployments respectively. The intellihiere-analyzer/app.py file acts as a Python microservice entry point when running outside Hugging Face.

7.2 API CONTRACTS AND REQUEST-RESPONSE FLOW

The primary contract for inference is the interview submission endpoint:

Endpoint: POST /api/interviews.

Request (simplified):

- Multipart form-data:
- Fields: transcription (string), question, questionId, jdId, questiontype, sessionduration, role, difficulty, skills, keywords (JSON array as string), additionalmetrics (optional JSON), useLLM (boolean) or

string "true"), source ("live", "jd", or "practice"), and optional jobDescription.

- Files: audio (optional audio file), video (optional video file).

Response (simplified):

- 201 Created with JSON containing:
 - message: status string.
 - sessionId, questionId, validated: flags summarizing JD and session linkage.
 - data: full InterviewResult document, including nested contentanalysis, voiceanalysis, facialanalysis, llmevaluation, comprehensivefeedback, and additionalmetrics.
 - analysisSummary: compact view with overallScore, multimodalScore, llmScore, contentScore, audioScore, videoScore, and integration flags.

The backend-to-ML contract (HF Space) is:

- **Endpoint (configurable):** POST HF_SPACE_URL + "/analyze".
- **Request:** Multipart form-data with keys audio and/or video (file streams), plus implicit authentication via Authorization: Bearer HF_SPACE_API_KEY if set.
- **Response:** JSON with success flag, audio and video feature dictionaries aligned with the schema expected by InterviewResult and backend scoring logic.

Analytics endpoints include /api/interviews/analytics, /api/interviews/ieee-summary, and percentile/compare-session routes, all implemented using aggregation pipelines over InterviewResult to compute mean scores, histograms, and IEEE-style confidence intervals via helper functions avg, stdSample, and ci95.

VIII. EXPERIMENTAL DESIGN

A preliminary pilot evaluation was conducted on 58 stored IntelliHire sessions; however, controlled teacher-labeled agreement analysis, full ablation studies, and latency benchmarking remain ongoing. [9]

8.1 EVALUATION PROTOCOL

A recommended protocol is:

1. **Dataset construction.** Collect a set of interview responses with synchronized audio, video, and transcripts using

IntelliHire’s three modes (Practice, JD Mode, Live), ensuring that metadata fields (role, difficulty, skills, JD traceability) are

quality and competencies (technical, clarity, confidence, engagement, conciseness), using the same scales as the LLM teacher; no labeling protocol is reported in the artifacts.

3. **Model training and selection.** Use ml-service/training_pipeline/train_chalearn_model.py and provided features.npy and labels.npy to train a confidence regressor; the underlying dataset is referred to as a Chalearn-style source[10], but specific details (size, split, identity) are not reported.
4. **Evaluation runs.** Run the system end-to-end under different configurations (Section 7.3) and compute metrics defined below, storing results in MongoDB and exporting via IEEEsession_analysis.py.

8.2 METRICS

Metrics consistent with the current analytics implementation include:

- **Accuracy-like score.** Correlation or mean absolute error between system overall scores and teacher ground-truth labels; this metric is defined by the framework but was not computed in the current pilot export. The current pilot dataset yielded an overall score mean of 49.6 with a 95% confidence interval of [45.3, 53.9].
- **Engagement score.** Average of normalized engagement proxies, such as facialanalysis.engagement or the canonical engagement metric combining multimodal and LLM signals in getAnalytics; operationalization is present in code but not summarized in a single scalar metric in the artifacts.
- **Confidence score.** The analyzer’s overall_confidence or backend-derived confidenceMetric in aggregations, capturing perceived confidence from audio-visual cues.[11]
- **Latency.** End-to-end time from frontend submission to backend response, decomposed into frontend upload, backend processing, multimodal analyzer latency, and LLM evaluation latency[4]; HF guides provide approximate expectations but no measured experimental distribution.
- **Usability.** Subjective user ratings of the feedback’s usefulness and clarity, potentially logged as additional metrics; no survey instrument is defined in the artifacts.[5]

populated; the exact dataset composition is not reported in the artifacts.

2. **Ground-truth labeling.** Obtain human teacher ratings per response for overall

8.3 BASELINES

The following baselines are aligned with the system’s modularity:

- **Content-only baseline.** Use overallContentScore as the primary score, ignoring multimodal and LLM components.
- **Multimodal-only baseline.** Use multimodalScore derived from content, audio, and video without any teacher or LLM scores.[11]
- **LLM-only baseline.** Use llmevaluation.overallScore or teacher evaluation scores as the sole predictor of ground-truth teacher labels.[7]
- **Heuristic-only vs. ML.** Compare analyzer outputs when using heuristic fusion versus when the classical ML regressor is available (i.e., confidence_model.pkl trained from features.npy/labels.npy).[10]
- **Local vs. Hugging Face deployment.** Compare latency and robustness when using local Python analyzer vs. HF Space (USE_LOCAL_ANALYZER=true vs. false) [9].

8.4 ABLATION PLAN

The ablation plan systematically toggles components to isolate their contributions:

- **Without heuristic fusion.** Use simple averaging of audio and video features or rely solely on the ML regressor for confidence, bypassing heuristic fusion and calibration[2]; implementation would require minor modifications to MultimodalConfidenceScorer (not present in artifacts).
- **Without LLM teacher.** Disable useLLM so that no LLM evaluations are incorporated; the backend then produces scores based solely on deterministic content and multimodal analyzers.[2], [3]
- **Without JD/RAG grounding.** Run evaluateResponse without providing JD or question rubrics, effectively using generic LLM feedback[1]; this configuration is not explicitly coded but can be realized by omitting JD and question IDs.
- **Local vs. HF Space analyzer.** Vary USE_LOCAL_ANALYZER

and HF_SPACE_URL to compare latency and robustness patterns.[4]

IX. PROPOSED EVALUATION FRAMEWORK AND PRELIMINARY PROTOCOL

9.1 Component-Wise Performance (Table A)

Table A. Component-wise performance across scoring stacks.

Table 3 - Pilot Statistical Summary

Metric	Value
Total sessions	58
Overall score (mean $\pm$ SD)	49.6 $\pm$ 16.6
95% CI	[45.3, 53.9]
JD source mean (n = 9)	45.4
Live source mean (n = 24)	42.0
Practice source mean (n = 25)	58.4
JD traceability	9/58 (15.5%)
Keyword relevance (mean $\pm$ SD)	50.0 $\pm$ 0.0
STAR method score (mean $\pm$ SD)	16.4 $\pm$ 25.9
Vocabulary diversity (mean $\pm$ SD)	91.2 $\pm$ 9.6
Coherence score (mean $\pm$ SD)	40.8 $\pm$ 19.6
Professional terminology (mean $\pm$ SD)	3.3 $\pm$ 6.7
Content score (mean $\pm$ SD)	29.4 $\pm$ 7.2

9.2 Planned Ablation Configurations. (Table B)

Table B. Ablation study for IntelliHire components.

Table 4 - Planned Ablation Configurations

Variant ID	Heuristic fusion	ML regressor	LLM teacher	JD/RAG grounding	Deployment (Local/HF)	Status / Notes
V1	Enabled	Enabled	Enabled	Enabled	Local	Reference configuration
V2	Disabled	Enabled	Enabled	Enabled	Local	Tests importance of heuristic calibration

Variant ID	Heuristic fusion	ML regressor	LLM teacher	JD/RAG grounding	Deployment (Local/HF)	Status / Notes
V3	Enabled	Disabled	Enabled	Enabled	Local	Tests ML vs heuristic fusion
V4	Enabled	Enabled	Disabled	Enabled	Local	No LLM contribution
V5	Enabled	Enabled	Enabled	Disabled	Local	No JD/RAG grounding
V6	Enabled	Enabled	Enabled	Enabled	HF Space	Tests deployment mode

9.3 Inference and Deployment Latency (Table C)

Table C. Inference and deployment latency breakdown.

Table 5 - Inference and Deployment Latency

Path	Expected warm latency (ms)	Expected cold-start latency (ms)	Status	Notes
Local analyzer	TBD	N/A	Not benchmarked	Python process on same host.
HF Space (CPU, warm)	8000–15000	N/A	Documentation-based expectation	Expected ~8–15 s for a 60 s clip; non-empirical.
HF Space (CPU, cold)	8000–15000	30000–60000	Documentation-based expectation	Free-tier cold start expected at ~30–60 s.
Backend end-to-end (full stack)	10000–20000	40000–80000	Documentation-based expectation	Includes network/upload overhead; implementation summary reports warm total ~10–20 s and cold total ~40–80 s per interview.

A preliminary pilot evaluation was conducted on 58 interview sessions recorded between February 21 and March 7, 2026. These sessions provide an initial descriptive summary of IntelliHire’s overall scoring behavior across Practice (n = 25), Live (n = 24), and JD-grounded (n = 9) modes. The reported tables summarize pilot score distributions, source-wise means, JD traceability, planned ablation settings, and documentation-based deployment timing expectations rather than fully controlled benchmark results.

X. DISCUSSION

The IntelliHire architecture demonstrates how deterministic analytics, multimodal signal processing, and generative AI can be combined into a deployable system suitable for academic study, with explicit hooks for JD grounding and IEEE-style statistics. The heuristic fusion logic and explicit schema design in InterviewResult provide interpretability and backward compatibility alongside the opaquer ML regressor and LLM teacher, enabling mixed-method evaluations of performance and user experience.

10.1 Strengths

Key strengths observable from the artifacts include:

- **Multimodality with traceability.** Audio and video features are explicitly logged and mapped to human-readable proxies (clarity, engagement, professionalism)[8], [11], supporting both per-session feedback and aggregated analytics across time, difficulty, and mode.
- **Grounded generative feedback.** The RAG-based LLM teacher uses job descriptions, resume text, and rubric points as context, and logs grounding metrics to quantify how

well feedback citations align with retrieved chunks.[1], [12]

- **Deployment flexibility.** The Hugging Face integration decouples heavy ML dependencies from the backend[9], with clear pathways for switching between local and cloud analyzers via environment variables and health checks.

10.2 Limitations

The current implementation also exhibits limitations:

- While this study provides initial empirical results from **58 sessions**, the sample size for JD-linked interviews (**n = 9**) remains small, necessitating further large-scale validation to generalize these findings.
- **Partially calibrated heuristics.** Some terms (e.g., speech rate score fixed at 0.7) are marked in comments as placeholders requiring empirical tuning, and configuration bounds (e.g., Config.ENERGY_MIN) lack data-driven justification in the artifacts.
- **Schema duplication and migration complexity.** The presence of both legacy field names (contentscore, overall_score) and enhanced snake_case fields (content_analysis, overall_score) indicates an ongoing migration that may complicate analytics if not carefully curated.

10.3 Bias and Fairness

Although not explicitly addressed in code or documentation, several aspects raise bias and fairness considerations:

- **Dataset bias.** The use of a Chlearn-style dataset for training the confidence regressor, without reported demographics or collection procedures, may encode biases in nonverbal behavior interpretation; the repository does not document mitigation strategies.[6]
- **Language and accent sensitivity.** Audio features such as energy and spectral descriptors, as well as LLM-based content evaluation, may systematically disadvantage speakers with diverse accents or non-standard prosody[2], but the artifacts do not contain fairness analyses.
- **Role and difficulty calibration.** While roles and difficulty labels are stored in InterviewResult, no code is present to adjust scoring baselines by role or difficulty, which may lead to inconsistent expectations across scenarios.[7]

10.4 Privacy Implications

The system stores audio paths, video paths, transcripts, and detailed analytics in MongoDB

collections, which raises privacy concerns for real candidates.[14] While the HF Space documentation notes that files processed in the Space are not stored and that no database is accessible from the Space, end-to-end data retention policies, anonymization strategies, and data subject rights are not specified in the artifacts. For academic deployment, institutional review and data handling protocols would need to be defined explicitly.[9], [10]

10.5 Threats to Validity

Given the absence of reported experiments, threats to validity are prospective but still important to articulate:

- **Internal validity.** Bugs or misalignments between analyzer outputs and stored schema fields (e.g., mixing legacy and enhanced field names) could confound analytics and cause misleading performance estimates; the presence of canonical metric expressions in getCanonicalMetricExpressions mitigates but does not eliminate this risk.
- **Construct validity.** Heuristic proxies such as eye contact and energy may not faithfully represent constructs like “confidence” or “engagement” across cultures and contexts; without user studies or expert annotation, the mapping remains unvalidated.[14], [15]
- **External validity.** Results obtained on self-selected student datasets or controlled demos may not transfer to industry interviews or high-stakes hiring contexts[6]; the artifacts do not describe target populations or deployment environments.

10.6 Ethics and Responsible AI

The IntelliHire system is positioned as a coaching and feedback tool rather than an automated decision-maker, which reduces but does not remove ethical risks[16]. Responsible deployment would require:

- Clearly communicating that scores and feedback are advisory and not used for hiring decisions.[12]
- Providing transparency about how features (audio, video, content) contribute to scores, which is facilitated by the explicit feature and metric logging in InterviewResult.[2]
- Allowing users to opt out of audio/video storage and providing mechanisms for data deletion, neither of which are specified in the current repository but can be layered on top of the existing API design.

XI. CONCLUSION AND FUTURE WORK

Within the constraints of the provided artifacts, IntelliHire offers a concrete blueprint for multimodal interview assessment that integrates

heuristic and classical ML confidence scoring[2], [10], deterministic transcript analytics, and RAG-based LLM feedback into a unified, deployable system. The validation of the system on **58 sessions** demonstrates that the proposed architecture effectively captures multimodal performance, with a notable strength in **Vocabulary Diversity (91.2)** and a identified need for coaching in **Professional Terminology (3.3)**. [2], [12]

Future work should prioritize data-driven calibration and validation of the heuristic and ML components, rigorous evaluation against human teacher labels, fairness and robustness analyses[9], and expansion of the RAG corpus to include institutional rubrics and best-practice guidelines.[12]

XII. REFERENCES

- [1] K. K. Jain, "AI Interviewer Chatbot for Technical and Hr Brilliance:A Tool for Upskilling Candidate," *International Journal of Engineering Research*, vol. 13, no. 02, 2024, doi: <https://doi.org/10.5281/zenodo.18170937>.
- [2] S. Yadav, Mr. B. Deshmukh, P. Kare, and O. Khaladkar, "AI-Driven Mock Interview: A New Era In Candidate Preparation," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 13, no. 11, Dec. 2024, doi: 10.17148/IJARCCCE.2024.131134.
- [3] J. Li *et al.*, "Listening to the Unspoken: Exploring '365' Aspects of Multimodal Interview Performance Assessment," Aug. 05, 2025, *arXiv*: arXiv:2507.22676. doi: 10.48550/arXiv.2507.22676.
- [4] Dr. V. Verma, R. Padwar, A. Chandrakar, K. Jaiswal, and P. Mishra, "AI-Powered Mock Interview System for Automated Skill Assessment," *IJRASET*, vol. 13, no. 11, pp. 2249–2253, Nov. 2025, doi: <https://doi.org/10.22214/ijraset.2025.75636>.
- [5] T. Daryanto, X. Ding, L. T. Wilhelm, S. Stil, K. M. Knutsen, and E. H. Rho, "Conversate: Supporting Reflective Learning in Interview Practice Through Interactive Simulation and Dialogic Feedback," *Proc. ACM Hum.-Comput. Interact.*, vol. 9, no. 1, pp. 1–32, Jan. 2025, doi: 10.1145/3701188.
- [6] A. Koutsoumpis *et al.*, "Beyond traditional interviews: Psychometric analysis of asynchronous video interviews for personality and interview performance evaluation using machine learning," *Computers in Human Behavior*, vol. 154, p. 108128, May 2024, doi: 10.1016/j.chb.2023.108128.
- [7] M. G A, P. S B, B. G B, Y. H P, Prof. B. Kumar B S, and Dr. H. S, "Development of an Intelligent Mock Interview Platform Using Generative AI," *IJIRT*, vol. 12, no. 7, Dec. 2025, doi: 10.64643/IJIRT1217-189276-459.
- [8] Prof. S. Kolpe, "AI Based Mock - Interview Behavioural Recognition Analyst," *IJRASET*, vol. 12, no. 5, pp. 3654–3666, May 2024, doi: 10.22214/ijraset.2024.61427.
- [9] A. B. Nofal *et al.*, "AI-enhanced interview simulation in the metaverse: Transforming professional skills training through VR and generative conversational AI," *Computers and Education: Artificial Intelligence*, vol. 8, p. 100347, Jun. 2025, doi: 10.1016/j.caeai.2024.100347.
- [10] V. S, S. V, S. Arunkumar, and A. AS, "AI-POWERED INTERVIEW COACHING: A MULTIMODAL FULL-STACK FRAMEWORK FOR BEHAVIORAL AND TECHNICAL ASSESSMENT," *IRJMETS*, doi: <https://www.doi.org/10.56726/IRJMETS81734>.
- [11] A. Agrawal, R. Anil George, S. S. Ravi, S. Kamath S, and A. Kumar, "Leveraging Multimodal Behavioral Analytics for Automated Job Interview Performance Assessment and Feedback," in *Second Grand-Challenge and Workshop on Multimodal Language (Challenge-HML)*, A. Zadeh, L.-P. Morency, P. P. Liang, and S. Poria, Eds., Seattle, USA: Association for Computational Linguistics, Jul. 2020, pp. 46–54. doi: 10.18653/v1/2020.challengehml-1.6.
- [12] A. Grattafiori *et al.*, "The Llama 3 Herd of Models," Nov. 23, 2024, *arXiv*: arXiv:2407.21783. doi: 10.48550/arXiv.2407.21783.
- [13] T. T. H. Nguyen, T. D. Q. Nguyen, H. L. Cao, T. C. T. Tran, T. C. M. Truong, and H. Cao, "SimInterview: Transforming Business Education through Large Language Model-Based Simulated Multilingual Interview Training System," Aug. 16, 2025, *arXiv*: arXiv:2508.11873. doi: 10.48550/arXiv.2508.11873.
- [14] P. Chavan and S. Jadhav, "A General Paper on AI Based Mock Interview System," *International Journal of Scientific Research in Science, Engineering and Technology*, vol. 13, no. 1, pp. 118–126, Jan. 2026, doi: 10.32628/IJSRSET26137.
- [15] W. Geng, C. Zhou, and Y. Bian, "Change gently: an agent-based virtual interview training for college students with great shyness," *Virtual Reality*, vol. 29, no. 1, p. 12, Dec. 2024, doi: 10.1007/s10055-024-01076-y.
- [16] S. Biswas, J.-Y. Jung, A. Unnam, K. Yadav, S. Gupta, and U. Gadiraju, "'Hi. I'm Molly, Your Virtual Interviewer!' -- Exploring the Impact of Race and Gender in AI-powered Virtual Interview Experiences," Aug. 26, 2024, *arXiv*: arXiv:2408.14159. doi: 10.48550/arXiv.2408.14159.

★ ★ ★